

| GUÍA 4: Aplicando soluciones con Kotlin

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1105	Desarrollo de Aplicaciones Móviles	EA1: Aplicando lenguajes de programacion para aplicaciones moviles
Tiempo	Modalidad de Trabajo	Indicadores de logro
2 horas	Parejas	IL1.2 IL 1.3

Antecedentes generales

Esta actividad busca que apliques los conceptos clave de la Programación Orientada a Objetos (POO) en Kotlin.
Su propósito es que diseñes e implementes un sistema simple de clases relacionadas, aplicando los principios de herencia, encapsulamiento y polimorfismo para crear un código bien estructurado, reutilizable y fácil de mantener.

Requerimientos para esta actividad

Para el desarrollo de esta actividad deberás disponer de:

- Conexión a internet
- IntelliJ IDEA
- JDK 17 o superior

Actividad

Deberás modelar un sistema simple de gestión de personal, siguiendo los ejemplos prácticos sugeridos en la presentación. Crearás una clase base Persona y una clase Empleado que herede de ella, aplicando los principios de la POO para organizar tu código.

Creación de la Clase Base Persona

1. Crea una clase llamada Persona. Las clases son plantillas que definen las propiedades y comportamientos de los objetos.
2. Añade las propiedades nombre (String) y edad (Int).
3. Crea un método llamado presentarse() que imprima en consola un saludo general, como por ejemplo: "Hola, mi nombre es [nombre].".
4. **Importante:** Marca la clase Persona con la palabra clave open para permitir que otras clases hereden de ella. Haz lo mismo con el método presentarse().



Implementación de Herencia con la Clase Empleado

1. Crea una nueva clase llamada Empleado que herede de Persona.
2. Añade dos propiedades nuevas y específicas para un empleado: puesto (String) y salario (Double).
3. Aplica el principio de **encapsulamiento**: haz que la propiedad salario sea private, de modo que solo sea accesible desde dentro de la misma clase, ayudando a mantener la integridad de los datos.
4. Crea un método público llamado mostrarPuesto() que imprima en consola un mensaje como: "[nombre] trabaja como [puesto].".

Aplicación de Polimorfismo

1. Dentro de la clase Empleado, **sobrescribe (override)** el método presentarse().
2. La nueva versión del método debe ser más específica, por ejemplo: "Hola, soy [nombre] y mi puesto es [puesto].".
3. Esto demuestra el **polimorfismo**, donde un método con el mismo nombre (presentarse) actúa de manera diferente dependiendo del objeto que lo llama.

Creación de Objetos y Verificación

1. En una función main(), crea dos **objetos** (instancias de clases):
 - Un objeto de la clase Persona.
 - Un objeto de la clase Empleado.
2. Llama al método presentarse() en ambos objetos.

Pregunta de reflexión

En un comentario en tu código, explica por qué la misma llamada al método presentarse() puede generar dos resultados distintos.

¿Qué concepto de la POO se está demostrando aquí? Argumenta con ejemplos